



HKBK COLLEGE OF ENGINEERING
(Affiliated to VTU, Belgaum and Approved by AICTE)
BANGALORE -560045
DEPARTMENT OF AIML

Machine Learning Laboratory
BAI606

As per VTU-Choice Based Credit System- 2022 Scheme
(Academic year: 2024 -2025)
6th Semester AIML

LABORATORY MANUAL

Prepared By

Prof. Vani. V
Prof. Bharath .V

Verified By

DQAC	HOD



HKBK COLLEGE OF ENGINEERING
(Affiliated to VTU, Belgaum and Approved by AICTE)
BANGALORE -560045
DEPARTMENT OF AIML

Vision and Mission of the Institution

Vision

To advance the intellectual capacity of the nation and the international community by imparting knowledge to graduates who are globally recognized as innovators, entrepreneur and competent professionals.

Mission

- M1.** To achieve academic excellence in science, Engineering, and technology through dedication to duty, innovation in teaching, and faith in human values.
- M2.** To enable our students to develop into outstanding professionals with high ethical standards to face the challenges of the 21st century.
- M3.** To provide educational opportunities to the deprived and weaker section of society to uplift their socioeconomic status.

Vision and Mission of the AIML Department

Vision

To advance the intellectual capacity of the nation and the international community by imparting knowledge to graduates who are globally recognized as innovators, entrepreneurs and competent professionals.

Mission

- M1.** To provide excellent technical knowledge and computing skills to make the graduates globally competitive with professional ethics.
- M2.** To involve in research activities and be committed to life long learning to make positive contributions to the society



HKBK COLLEGE OF ENGINEERING
(Affiliated to VTU, Belgaum and Approved by AICTE)
DEPARTMENT OF AIML

Programme Outcomes (PO's)/Graduate Attributes (GA)	
PO1	Engineering Knowledge: Apply knowledge of mathematics, science, engineering Fundamentals and an engineering specialization to the solution of complex engineering problems.
PO2	Problem Analysis: Identity, formulate, research literature and analyze complex engineering problems reaching substantiated conclusions using the first principles of mathematics, natural sciences and engineering sciences.
PO3	Design/ Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, and cultural, societal and environmental considerations.
PO4	Conduct investigations of complex problems using research-based knowledge And research methods including design of experiments, analysis, and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modelling to complex engineering activities with an under-standing of the limitations.
PO6	The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.
PO7	Environment and Sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate Knowledge of and need for sustainable development.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO9	Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective Presentations and give and receive clear instructions.
PO11	Project Management and Finance: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long Learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Programme Specific Outcomes (PSO's)	
PSO1	Problem-Solving Skills: An ability to investigate and solve a problem by analysis, interpretation of data, design, and implementation through appropriate techniques, tools, and skills.
PSO2	Professional Skills: An ability to apply algorithmic principles, computing skills and computer science theory in the modelling and design of computer-based systems.
PSO3	Entrepreneurial Ability: An ability to apply design, development principles, and management skills in the construction of software products of varying complexity to Become an entrepreneur

Programme Educational Objectives (PEO's)	
PEO-1	To provide students with a strong foundation in engineering fundamentals and in the AIML to work in the global scenario.
PEO-2	To provide a sound knowledge of programming and computing techniques and good communication and interpersonal skills so that they will be capable of analysing, designing, and building innovative software systems.
PEO-3	To equip students in their chosen field of engineering and related fields to enable them to work in multidisciplinary teams.
PEO-4	To inculcate in students professional, personal, and ethical attitudes to relate engineering issues to broader social context and become a responsible citizens.
PEO-5	To provide students with an environment for life-long learning which allows them to successfully adapt to the evolving technologies throughout their professional Carrier and face global challenges.

Mapping of Course Outcomes to Program Outcomes

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	1	1	1	1	2	-	-	-	-	-	1	1	1	-	-
CO2	3	2	1	-	3	-	-	-	-	-	1	2	2	3	1
CO3	2	1	1	1	2	-	-	-	-	-	1	1	1	2	-
CO4	2	1	1	1	2	-	-	-	-	-	1	3	1	2	-
CO5	3	2	1	2	3	-	-	-	-	-	1	2	2	3	-
AVG	2.2	1.4	1	1.25	2.40	-	-	-	-	-	1	1.80	1.40	2.50	1

3	High	2	Moderate	1	Low	-	No Correlation
---	------	---	----------	---	-----	---	----------------

MACHINE LEARNING LABORATORY SEMESTER – VI

Course Code	BAI606	CIE Marks	50
Number of Contact Hours/Week	0:0:2:0	SEE Marks	50
Credits	01	Exam Hours	02

Course Learning Objectives: This course (BCSL606) will enable students to:

- To become familiar with data and visualize univariate, bivariate, and multivariate data using statistical techniques and dimensionality reduction.
- To understand various machine learning algorithms such as similarity-based learning, regression, decision trees, and clustering.
- To familiarize with learning theories, probability-based models and developing the skills required for Decision-making in dynamic environments.

Descriptions(if any):

The installation procedure of the required software must be demonstrated, carried out in groups, and documented in the journal.

Programs List:

1.	Develop a program to create histograms for all numerical features and analyze the distribution of each feature. Generate box plots for all numerical features and identify any outliers. Use California Housing dataset.
2.	Develop a program to Compute the correlation matrix to understand the relationships between pairs of features. Visualize the correlation matrix using a heat map to know which variables have strong positive/negative correlations. Create a pair plot to visualize pair wise relationships between features. Use California Housing dataset.
3.	Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.
4.	For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.
5.	Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of $[0,1]$. Perform the following based on dataset generated. a. Label the first 50 points $\{x_1, \dots, x_{50}\}$ as follows: if $(x_i \leq 0.5)$, then $x_i \in \text{Class}_1$, else $x_i \in \text{Class}_2$ b. Classify the remaining points, $x_{51}, \dots, x_{100}$ using KNN. Perform this for $k=1, 2, 3, 4, 5, 20, 30$
6.	Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.
7.	Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset (for vehicle fuel efficiency prediction) for Polynomial Regression.
8.	Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.
9.	Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets.
10.	Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

Laboratory Outcomes: The student should be able to:

- Illustrate the principles of multivariate data and apply dimensionality reduction techniques.
- Demonstrate similarity-based learning methods and perform regression analysis.
- Develop decision trees for classification and regression problems, and Bayesian models for probabilistic learning.
- Implement the clustering algorithms to share computing resources.

Assessment Details (both CIE and SEE): The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks out of 50) and for the SEE minimum passing mark is 35% of the maximum marks (18 out of 50 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together

Continuous Internal Evaluation (CIE):

CIE marks for the practical course are **50 Marks**.

The split-up of CIE marks for record/ journal and test are in the ratio **60:40**.

- Each experiment is to be evaluated for conduction with an observation sheet and record write-up. Rubrics for the evaluation of the journal/write-up for hardware/software experiments are designed by the faculty who is handling the laboratory session and are made known to students at the beginning of the practical session.
- Record should contain all the specified experiments in the syllabus and each experiment write-up will be evaluated for 10 marks.
- Total marks scored by the students are scaled down to **30 marks** (60% of maximum marks).
- Weightage to be given for neatness and submission of record/write-up on time.
- Department shall conduct a test of 100 marks after the completion of all the experiments listed in the syllabus.
- In a test, test write-up, conduction of experiment, acceptable result, and procedural knowledge will carry a weightage of 60% and the rest 40% for viva-voce.
- The suitable rubrics can be designed to evaluate each student's performance and learning ability.
- The marks scored shall be scaled down to **20 marks** (40% of the maximum marks).

The Sum of scaled-down marks scored in the report write-up/journal and marks of a test is the total CIE marks scored by the student.

Semester End Evaluation (SEE):

- SEE marks for the practical course are 50 Marks.
- SEE shall be conducted jointly by the two examiners of the same institute, examiners are appointed by the Head of the Institute.
- The examination schedule and names of examiners are informed to the university before the conduction of the examination. These practical examinations are to be conducted between the schedule mentioned in the academic calendar of the University.
- All laboratory experiments are to be included for practical examination.
- (Rubrics) Breakup of marks and the instructions printed on the cover page of the answer script to be strictly adhered to by the examiners. **OR** based on the course requirement evaluation rubrics shall be decided jointly by examiners.
- Students can pick one question (experiment) from the questions lot prepared by the examiners jointly.
- Evaluation of test write-up/ conduction procedure and result/viva will be conducted jointly by examiners.

General rubrics suggested for SEE are mentioned here, writeup-20%, Conduction procedure and result in -60%, Viva-voce 20% of maximum marks. SEE for practical shall be evaluated for 100 marks and scored marks shall be scaled down to 50 marks (however, based on course type, rubrics shall be decided by the examiners)

Change of experiment is allowed only once and 15% of Marks allotted to the procedure part are to be made zero.

The minimum duration of SEE is 02 hours

Program 1

1. Develop a program to create histograms for all numerical features and analyse the distribution of each feature. Generate box plots for all numerical features and identify any outliers. Use California Housing dataset.

PROGRAM:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Load the California Housing dataset from a local file or a CSV
# You can replace the file path with your actual file path
# Example: housing_df = pd.read_csv('path_to_your_file.csv')
housing_df = pd.read_csv(r'C:\Users\Students\Desktop\mohazzeba\Datasets\housing.csv')

# If the dataset is already loaded into housing_df, skip the file loading step

# Step 2: Create histograms for numerical features
numerical_features = housing_df.select_dtypes(include=[np.number]).columns

# Plot histograms
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numerical_features):
    plt.subplot(3, 3, i + 1)
    sns.histplot(housing_df[feature], kde=True, bins=30, color='blue')
    plt.title(f'Distribution of {feature}')
plt.tight_layout()
plt.show()

# Step 3: Generate box plots for numerical features
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numerical_features):
    plt.subplot(3, 3, i + 1)
    sns.boxplot(x=housing_df[feature], color='orange')
    plt.title(f'Box Plot of {feature}')
plt.tight_layout()
plt.show()

# Step 4: Identify outliers using the IQR method
print("Outliers Detection:")
outliers_summary = {}
for feature in numerical_features:
    Q1 = housing_df[feature].quantile(0.25)
    Q3 = housing_df[feature].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = housing_df[(housing_df[feature] < lower_bound) | (housing_df[feature] > upper_bound)]
```

```

outliers_summary[feature] = len(outliers)
print(f'{feature}: {len(outliers)} outliers')

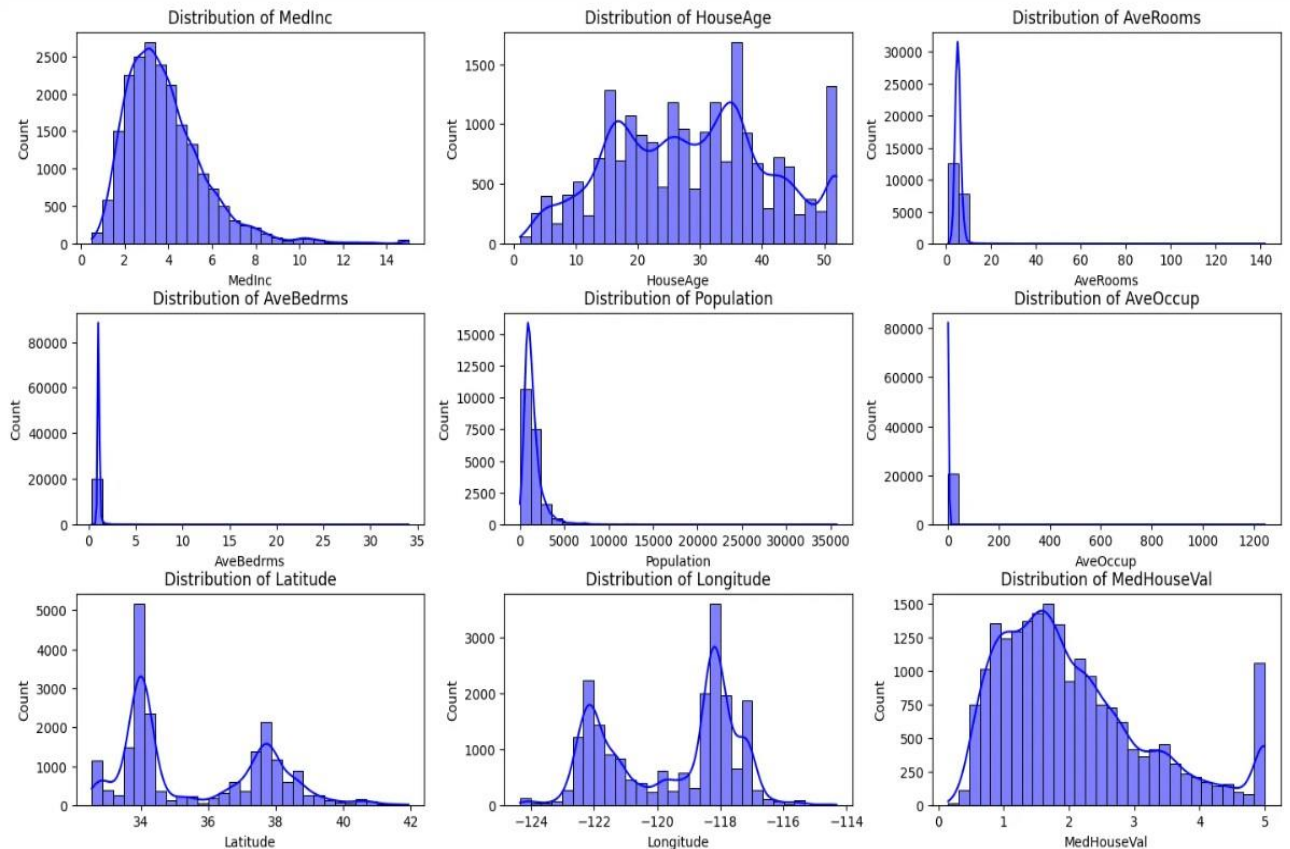
```

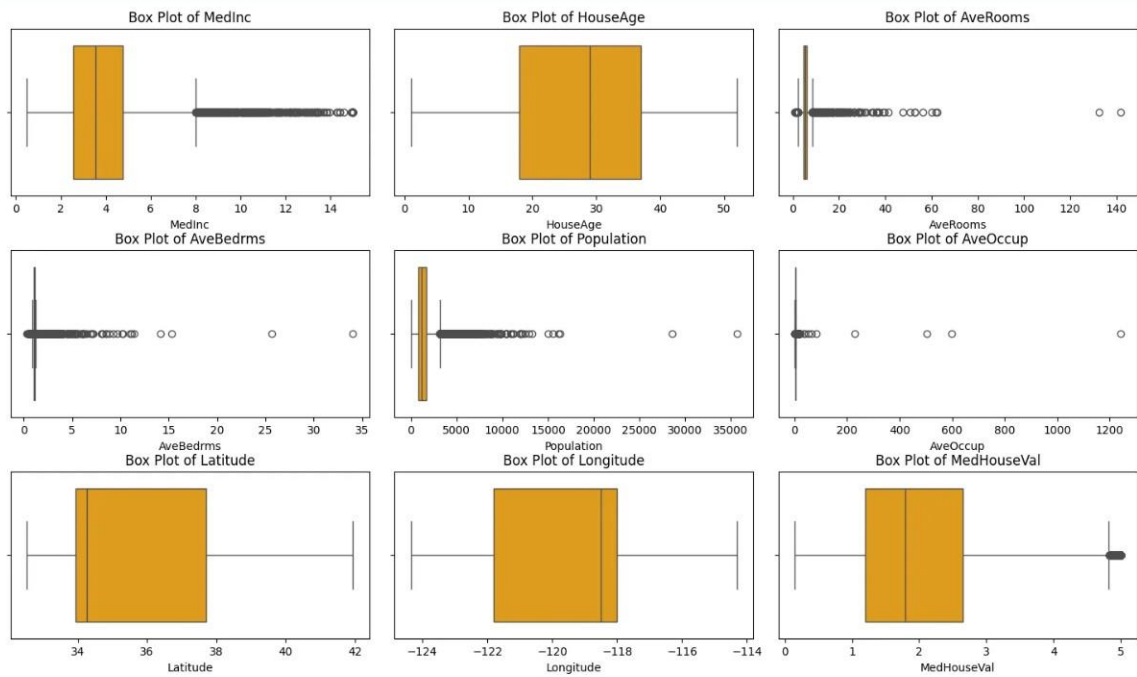
```

# Optional: Print a summary of the dataset
print("\nDataset Summary:")
print(housing_df.describe())

```

Output





2. Develop a program to Compute the correlation matrix to understand the relationships between pairs of features. Visualize the correlation matrix using a heatmap to know which variables have strong positive/negative correlations. Create a pair plot to visualize pairwise relationships between features. Use California Housing dataset.

```
# Import necessary libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset from a CSV file in the folder (provide your file path)
file_path = r'C:\Users\Students\Desktop\mohazzeba\Datasets\housing.csv'
df = pd.read_csv(file_path)

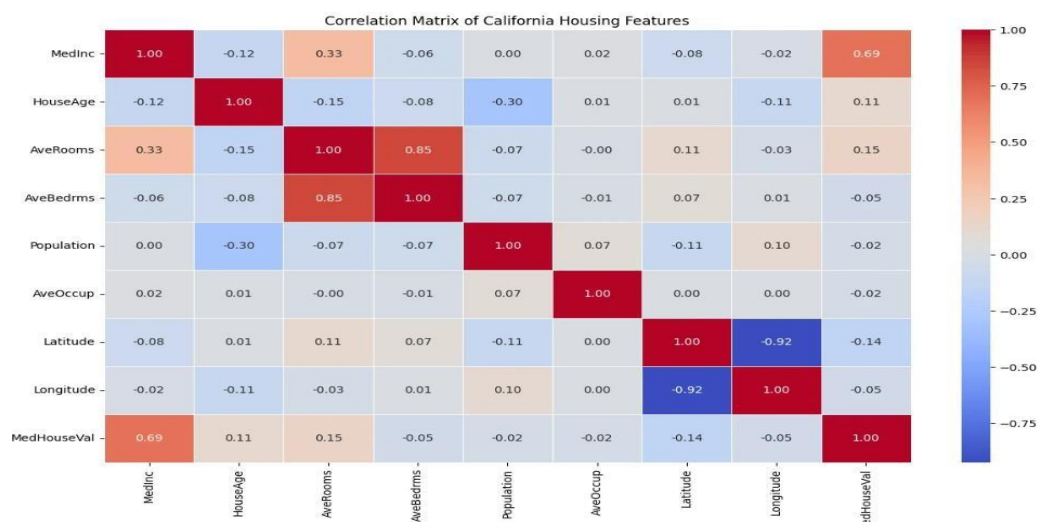
# Check the first few rows of the dataset
print(df.head())

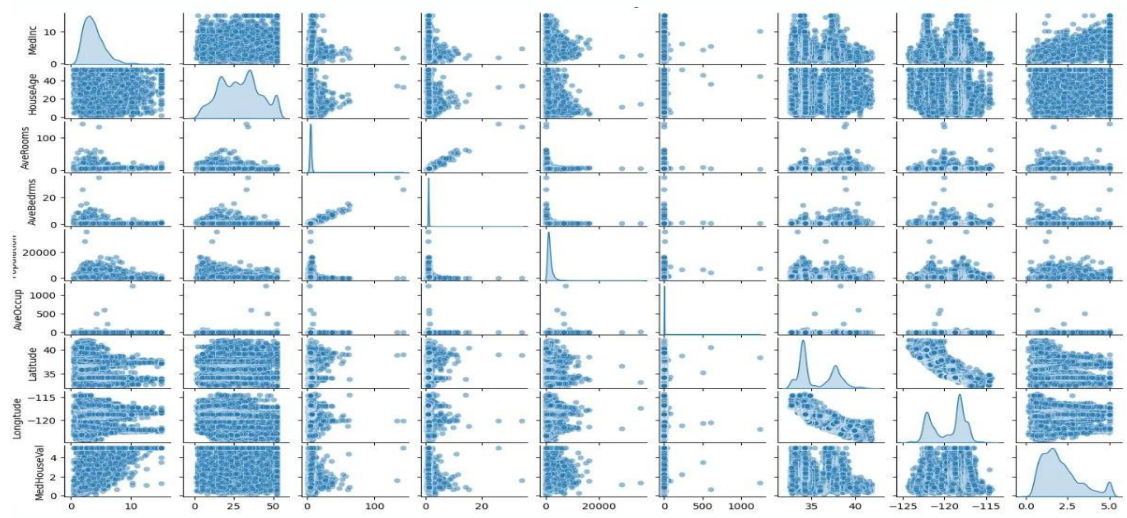
# Compute the correlation matrix
corr_matrix = df.corr()

# Plot the correlation matrix using a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title("Correlation Matrix of California Housing Dataset")
plt.show()

# Create a pair plot to visualize pairwise relationships between features
sns.pairplot(df, height=2.5)
plt.suptitle("Pairwise Relationships of Features", y=1.02)
plt.show()
```

OUTPUT:





3. Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Step 1: Load the Iris dataset from a local file (replace with your file path if necessary)
# Example: iris_df = pd.read_csv('path_to_iris_dataset.csv')
# iris_df = pd.read_csv('iris.csv') # Assuming you've saved the Iris dataset as a CSV file

# If you already have the dataset as a pandas DataFrame, use the following:
# Assuming the data has columns 'sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
# 'petal width (cm)'
# and the target labels are stored separately (for example, 'species')

# Sample DataFrame structure (use your own dataset loading method)
iris_df = pd.read_csv(r'C:\Users\Students\Desktop\mohazzeba\Datasets\iris.csv') # Replace
this with actual path to the CSV

# Step 2: Extract feature data and labels
data = iris_df.drop('species', axis=1) # Replace 'species' with your actual target column name
labels = iris_df['species'] # Replace with your target column name, if necessary
label_names = labels.unique()

# Step 3: Perform PCA to reduce dimensionality to 2
pca = PCA(n_components=2)
data_reduced = pca.fit_transform(data)

# Step 4: Create a DataFrame for the reduced data
reduced_df = pd.DataFrame(data_reduced, columns=['Principal Component 1', 'Principal
Component 2'])
reduced_df['Label'] = labels

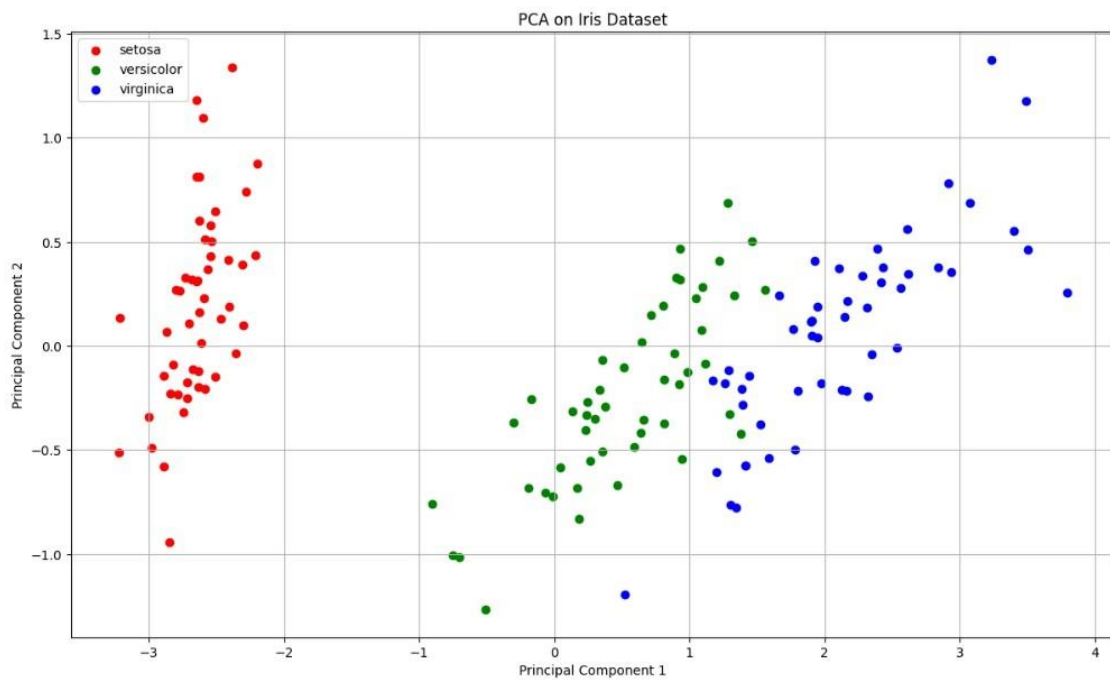
# Step 5: Plot the reduced data
plt.figure(figsize=(8, 6))
colors = ['r', 'g', 'b']

for i, label in enumerate(np.unique(labels)):
    plt.scatter(
        reduced_df[reduced_df['Label'] == label]['Principal Component 1'],
        reduced_df[reduced_df['Label'] == label]['Principal Component 2'],
        label=label_names[i], # This will map label to the target names
        color=colors[i]
    )

plt.title('PCA on Iris Dataset')
plt.xlabel('Principal Component 1')
```

```
plt.ylabel('Principal Component 2')  
plt.legend()  
plt.grid()  
plt.show()
```

OUTPUT:



4. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.

```
import pandas as pd
def find_s_algorithm(file_path):
    data = pd.read_csv(file_path)

    print("Training data:")
    print(data)

    attributes = data.columns[:-1]
    class_label = data.columns[-1]

    hypothesis = ['?' for _ in attributes]

    for index, row in data.iterrows():
        if row[class_label] == 'Yes':
            for i, value in enumerate(row[attributes]):
                if hypothesis[i] == '?' or hypothesis[i] == value:
                    hypothesis[i] = value
                else:
                    hypothesis[i] = '?'

    return hypothesis

file_path = (r'C:\Users\Students\Desktop\mohazzeba\Datasets\training_data.csv')
hypothesis = find_s_algorithm(file_path)
print("\nThe final hypothesis is:", hypothesis)
```

OUTPUT:

Training data:

	Outlook	Temperature	Humidity	Windy	PlayTennis
0	Sunny	Hot	High	False	No
1	Sunny	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Rain	Cold	High	False	Yes
4	Rain	Cold	High	True	No
5	Overcast	Hot	High	True	Yes
6	Sunny	Hot	High	False	No

The final hypothesis is: ['Overcast', 'Hot', 'High', '?']

5. Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of [0,1]. Perform the following based on dataset generated.

a) Label the first 50 points $\{x_1, \dots, x_{50}\}$ as follows: if $(x_i \leq 0.5)$, then $x_i \in \text{Class1}$, else $x_i \in \text{Class1}$

b) Classify the remaining points, $x_{51}, \dots, x_{100}$ using KNN. Perform this for $k=1,2,3,4,5,20,30$

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
```

```
data = np.random.rand(100)
```

```
labels = ["Class1" if x <= 0.5 else "Class2" for x in data[:50]]
```

```
def euclidean_distance(x1, x2):
    return abs(x1 - x2)
```

```
def knn_classifier(train_data, train_labels, test_point, k):
    distances = [(euclidean_distance(test_point, train_data[i]), train_labels[i]) for i in
range(len(train_data))]
```

```
distances.sort(key=lambda x: x[0])
k_nearest_neighbors = distances[:k]
```

```
k_nearest_labels = [label for _, label in k_nearest_neighbors]
```

```
    return Counter(k_nearest_labels).most_common(1)[0][0]
```

```
train_data = data[:50]
train_labels = labels
```

```
test_data = data[50:]
```

```
k_values = [1, 2, 3, 4, 5, 20, 30]
```

```
print("--- k-Nearest Neighbors Classification ---")
print("Training dataset: First 50 points labeled based on the rule (x <= 0.5 -> Class1, x > 0.5 -> Class2)")
print("Testing dataset: Remaining 50 points to be classified\n")
```

```
results = {}
```

```
for k in k_values:
    print(f'Results for k = {k}:')
```

```

classified_labels = [knn_classifier(train_data, train_labels, test_point, k) for test_point in
test_data]
    results[k] = classified_labels

    for i, label in enumerate(classified_labels, start=51):
print(f'Point x{i} (value: {test_data[i - 51]:.4f}) is classified as {label}')
    print("\n")

print("Classification complete.\n")

for k in k_values:
classified_labels = results[k]
    class1_points = [test_data[i] for i in range(len(test_data)) if classified_labels[i] ==
"Class1"]
    class2_points = [test_data[i] for i in range(len(test_data)) if classified_labels[i] ==
"Class2"]

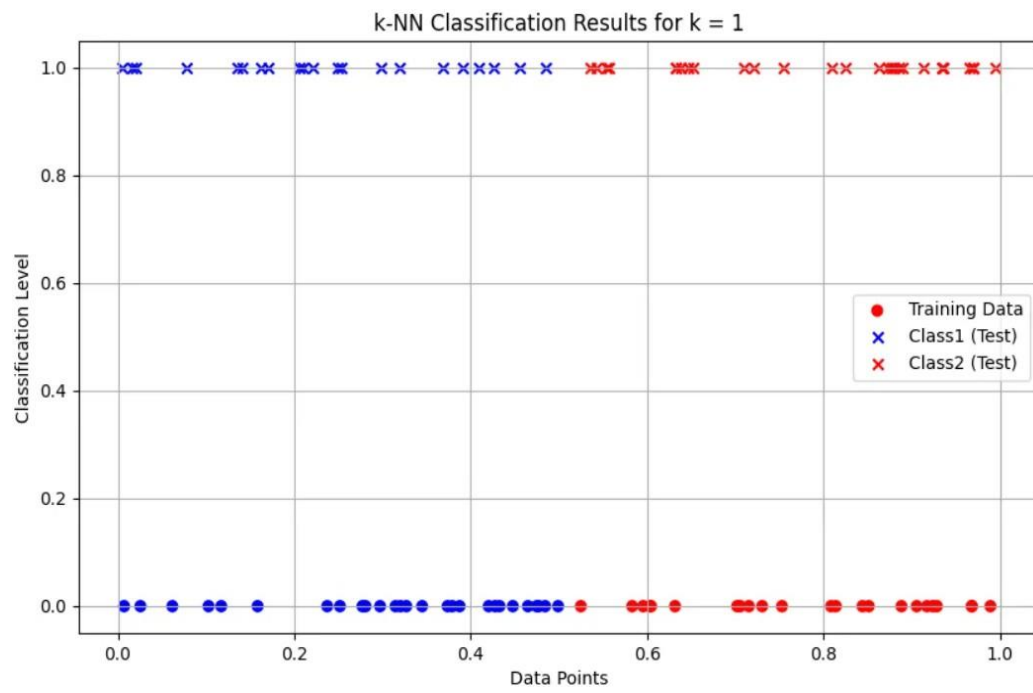
plt.figure(figsize=(10, 6))

plt.scatter(train_data, [0] * len(train_data), c=["blue" if label == "Class1" else "red" for label
in train_labels],
            label="Training Data", marker="o")
plt.scatter(class1_points, [1] * len(class1_points), c="blue", label="Class1 (Test)",
marker="x")
plt.scatter(class2_points, [1] * len(class2_points), c="red", label="Class2 (Test)",
marker="x")

plt.title(f'k-NN Classification Results for k = {k}')
plt.xlabel("Data Points")
plt.ylabel("ClassificationLevel")
plt.legend()
plt.grid(True)
plt.show()

```


OUTPUT:



6. Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
def gaussian_kernel(x, xi, tau):

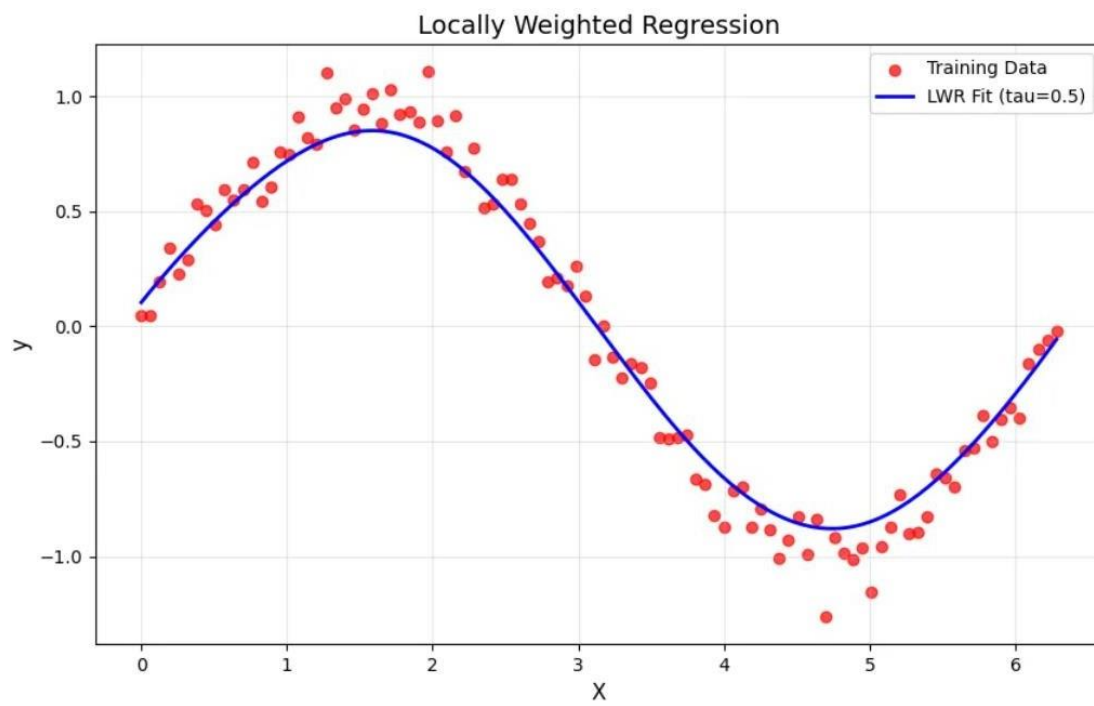
    return np.exp(-np.sum((x - xi) ** 2) / (2 * tau ** 2))
def locally_weighted_regression(x, X, y, tau):
    m = X.shape[0]

    weights = np.array([gaussian_kernel(x, X[i], tau) for i in range(m)])
    W = np.diag(weights)
    X_transpose_W = X.T @ W
    theta = np.linalg.inv(X_transpose_W @ X) @ X_transpose_W @ y
    return x @ theta

np.random.seed(42)
X = np.linspace(0, 2 * np.pi, 100)
y = np.sin(X) + 0.1 * np.random.randn(100)
X_bias = np.c_[np.ones(X.shape), X]
x_test = np.linspace(0, 2 * np.pi, 200)
x_test_bias = np.c_[np.ones(x_test.shape), x_test]
tau = 0.5

y_pred = np.array([locally_weighted_regression(xi, X_bias, y, tau) for xi in x_test_bias])
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='red', label='Training Data', alpha=0.7)
plt.plot(x_test, y_pred, color='blue', label=f'LWR Fit (tau={tau})', linewidth=2)
plt.xlabel('X', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.title('Locally Weighted Regression', fontsize=14)
plt.legend(fontsize=10)
plt.grid(alpha=0.3)
plt.show()
```

OUTPUT:



7. Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset (for vehicle fuel efficiency prediction) for Polynomial Regression.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Load datasets (replace the paths with your local paths)
boston_df = pd.read_csv(r'C:\Users\Students\Desktop\mohazzeba\Datasets\BostonHousing.csv')
auto_mpg_df = pd.read_csv(r'C:\Users\Students\Desktop\mohazzeba\Datasets\auto-mpg.csv')

# --- Step 1: Data Preprocessing ---
# Handle NaN values and remove rows with NaN values
boston_df = boston_df.dropna()
auto_mpg_df = auto_mpg_df.dropna()

# --- Step 2: Linear Regression on Boston Housing Dataset ---
# Features and target variable for Boston Housing dataset
X_boston = boston_df[['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
'PTRATIO', 'B', 'LSTAT']]
y_boston = boston_df['MEDV']

# Split the data into training and testing sets
X_train_boston, X_test_boston, y_train_boston, y_test_boston = train_test_split(X_boston, y_boston,
test_size=0.2, random_state=42)

# Linear Regression Model
lin_reg_boston = LinearRegression()
lin_reg_boston.fit(X_train_boston, y_train_boston)

# Predictions on the test set
y_pred_boston = lin_reg_boston.predict(X_test_boston)

# Calculate Mean Squared Error
mse_boston = mean_squared_error(y_test_boston, y_pred_boston)
print(f"Linear Regression MSE on Boston Housing Dataset: {mse_boston}")

# Plot Actual vs Predicted for Boston Housing
plt.figure(figsize=(8, 6))
plt.scatter(y_test_boston, y_pred_boston)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Linear Regression: Actual vs Predicted (Boston Housing)')
plt.show()

# --- Step 3: Polynomial Regression on Auto MPG Dataset ---
# Features and target variable for Auto MPG dataset
```

```

X_auto_mpg = auto_mpg_df[['Cylinders', 'Displacement', 'Horsepower', 'Weight', 'Acceleration',
'Model Year']]
y_auto_mpg = auto_mpg_df['MPG']

# Split the data into training and testing sets
X_train_auto_mpg, X_test_auto_mpg, y_train_auto_mpg, y_test_auto_mpg =
train_test_split(X_auto_mpg, y_auto_mpg, test_size=0.2, random_state=42)

# Apply Polynomial Features (degree 2 for this example)
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train_auto_mpg)
X_test_poly = poly.transform(X_test_auto_mpg)

# Polynomial Regression Model
poly_reg = LinearRegression()
poly_reg.fit(X_train_poly, y_train_auto_mpg)

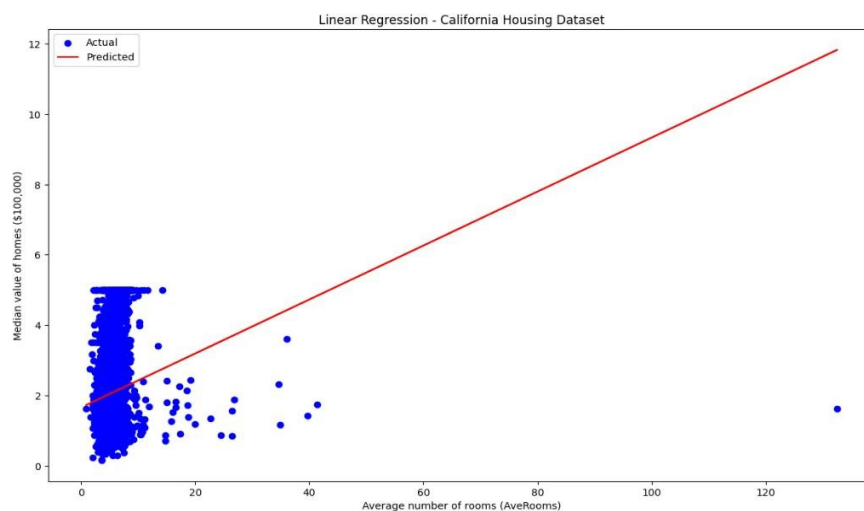
# Predictions on the test set
y_pred_auto_mpg = poly_reg.predict(X_test_poly)

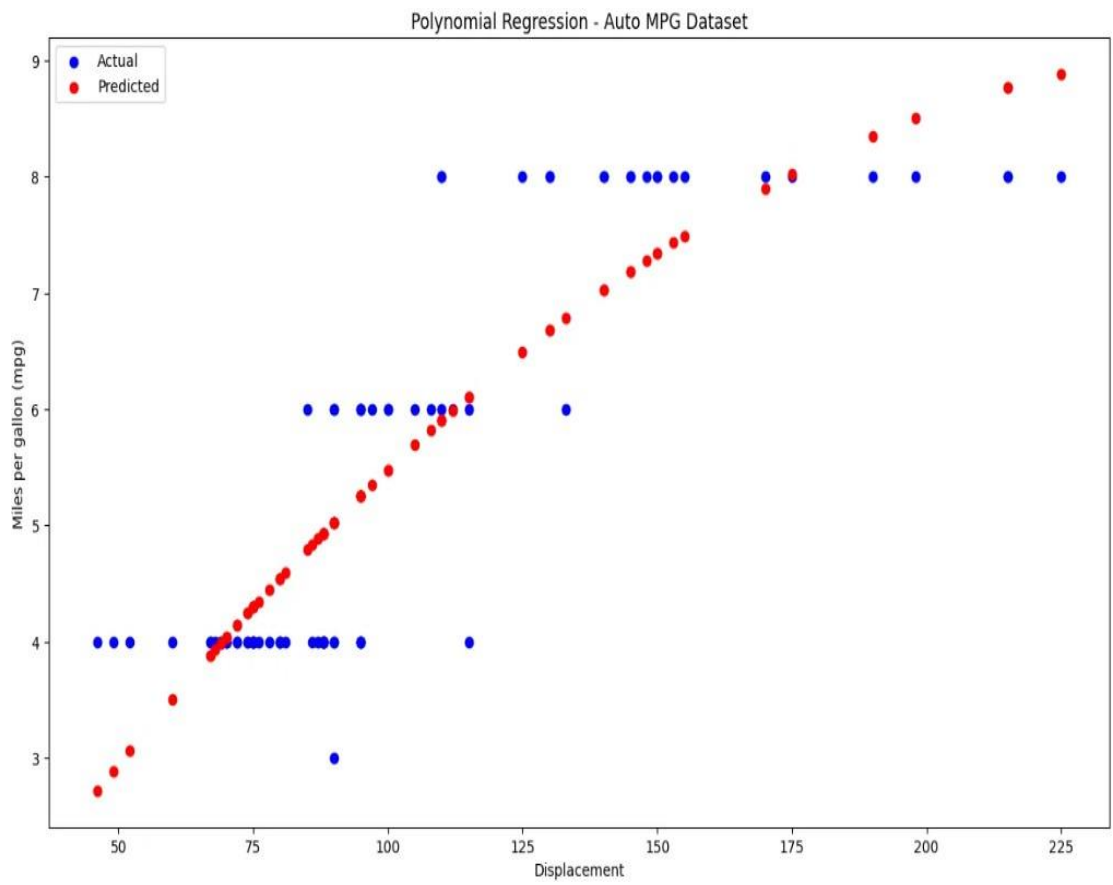
# Calculate Mean Squared Error
mse_auto_mpg = mean_squared_error(y_test_auto_mpg, y_pred_auto_mpg)
print(f"Polynomial Regression MSE on Auto MPG Dataset: {mse_auto_mpg}")

# Plot Actual vs Predicted for Auto MPG
plt.figure(figsize=(8, 6))
plt.scatter(y_test_auto_mpg, y_pred_auto_mpg)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Polynomial Regression: Actual vs Predicted (Auto MPG)')
plt.show()

```

OUTPUT:





8. Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Step 1: Load the Breast Cancer dataset
# Load the dataset from your folder (replace with your local path)
data = pd.read_csv(r'C:\Users\Students\Desktop\mohazzeba\Datasets\BreastCancer.csv')

# Step 2: Preprocess the data (Handle missing values and encoding)
# Here, we assume that the target variable is in the last column
# and that the dataset doesn't have missing values.
X = data.iloc[:, :-1] # Features (all columns except the last one)
y = data.iloc[:, -1] # Target (the last column)

# Handle missing values if present
X = X.fillna(X.mean()) # Fill missing values with the mean (or other strategies)

# Step 3: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 4: Build the Decision Tree classifier
decision_tree = DecisionTreeClassifier(random_state=42)
decision_tree.fit(X_train, y_train)

# Step 5: Make predictions
y_pred = decision_tree.predict(X_test)

# Step 6: Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy of the Decision Tree classifier: {accuracy:.4f}')
```

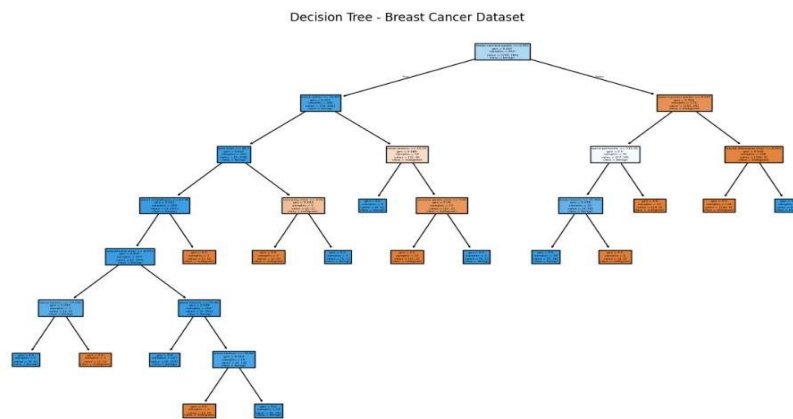
```
# Step 7: Visualize the decision tree (optional)
plt.figure(figsize=(12, 8))
plot_tree(decision_tree, filled=True, feature_names=X.columns,
class_names=np.unique(y).astype(str))
plt.title('Decision Tree Visualization')
plt.show()
```

```
# Step 8: Classify a new sample
```

```
# Define a new sample (replace with actual values that you want to classify)
# Example: [mean radius, mean texture, mean perimeter, mean area, etc.]
new_sample = np.array([[15.6, 19.0, 98.0, 650.0, 0.2, 0.5, 0.3, 0.1, 0.7, 0.6]]) # Example
input

# Classify the new sample using the trained model
new_sample_pred = decision_tree.predict(new_sample)
print(f"The predicted class for the new sample is: {new_sample_pred[0]}")
```

OUTPUT:



9. Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets.

PROGRAM:

```
import numpy as np
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt

data = fetch_olivetti_faces(shuffle=True, random_state=42)
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy * 100:.2f}%')

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=1))

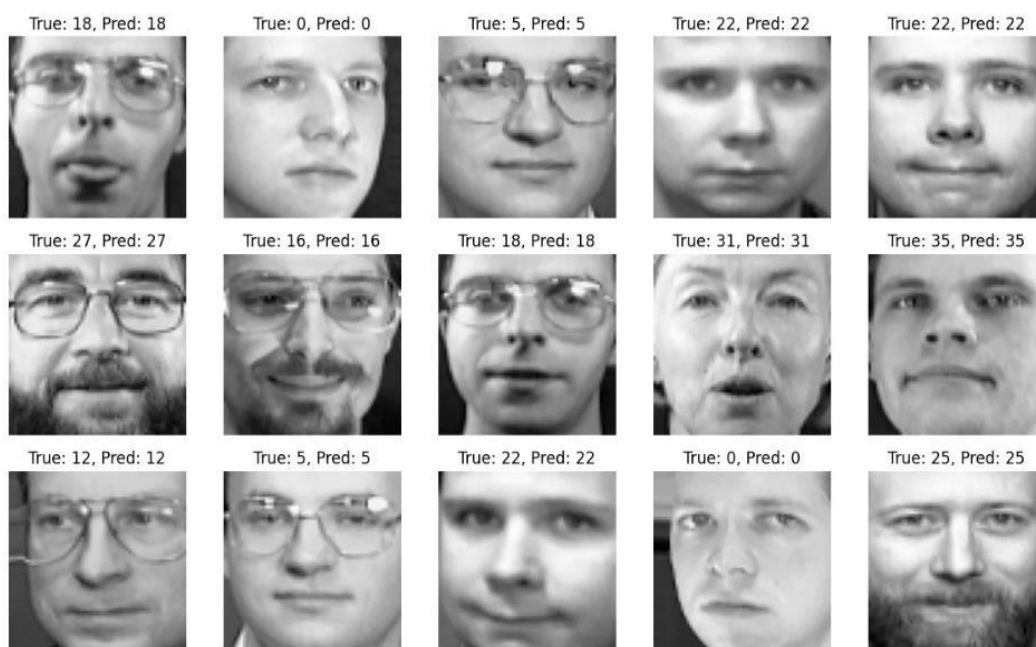
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

cross_val_accuracy = cross_val_score(gnb, X, y, cv=5, scoring='accuracy')
print(f'\nCross-validation accuracy: {cross_val_accuracy.mean() * 100:.2f}%')

fig, axes = plt.subplots(3, 5, figsize=(12, 8))
for ax, image, label, prediction in zip(axes.ravel(), X_test, y_test, y_pred):
    ax.imshow(image.reshape(64, 64), cmap=plt.cm.gray)
    ax.set_title(f'True: {label}, Pred: {prediction}')
    ax.axis('off')

plt.show()
```

OUTPUT:



10. Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
import warnings
warnings.filterwarnings('ignore')

# Load the Wisconsin Breast Cancer Dataset from a local CSV file
dataset_path = r'C:\Users\Students\Desktop\mohazzeba\Datasets\Wisconsin Breast Cancer dataset.csv'
df = pd.read_csv(dataset_path)

# Drop unnecessary columns if present
if 'id' in df.columns:
    df.drop(columns=['id'], inplace=True)
if 'Unnamed: 32' in df.columns:
    df.drop(columns=['Unnamed: 32'], inplace=True)

# Convert categorical diagnosis column to numerical
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Separate features from target variable
df.drop(columns=['diagnosis'], inplace=True)

# Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# Reduce to 2 dimensions using PCA for visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Determine optimal number of clusters using the Elbow Method
```

```

wcss = []
K_range = range(1, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)

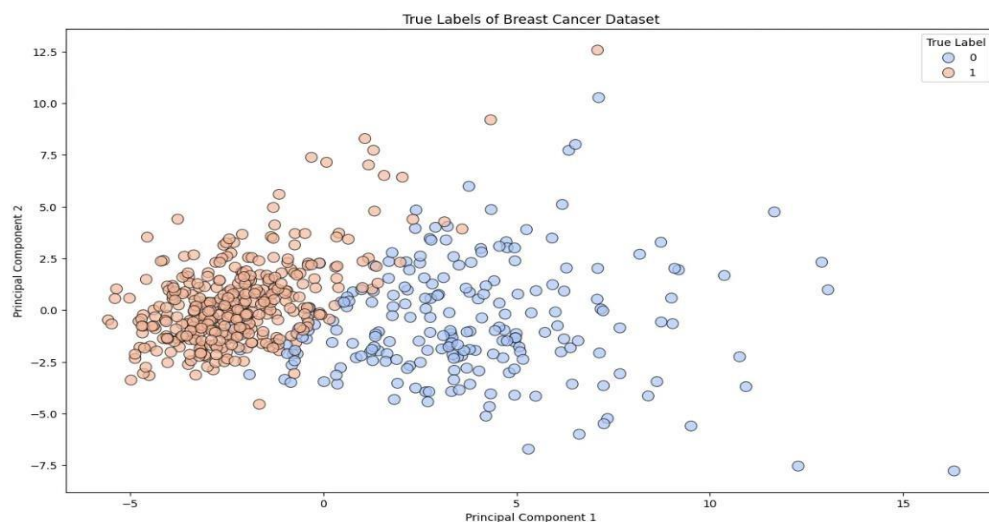
# Plot the Elbow Method Graph
plt.figure(figsize=(8, 5))
plt.plot(K_range, wcss, marker="o", linestyle="-")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("WCSS")
plt.title("Elbow Method to Find Optimal k")
plt.show()

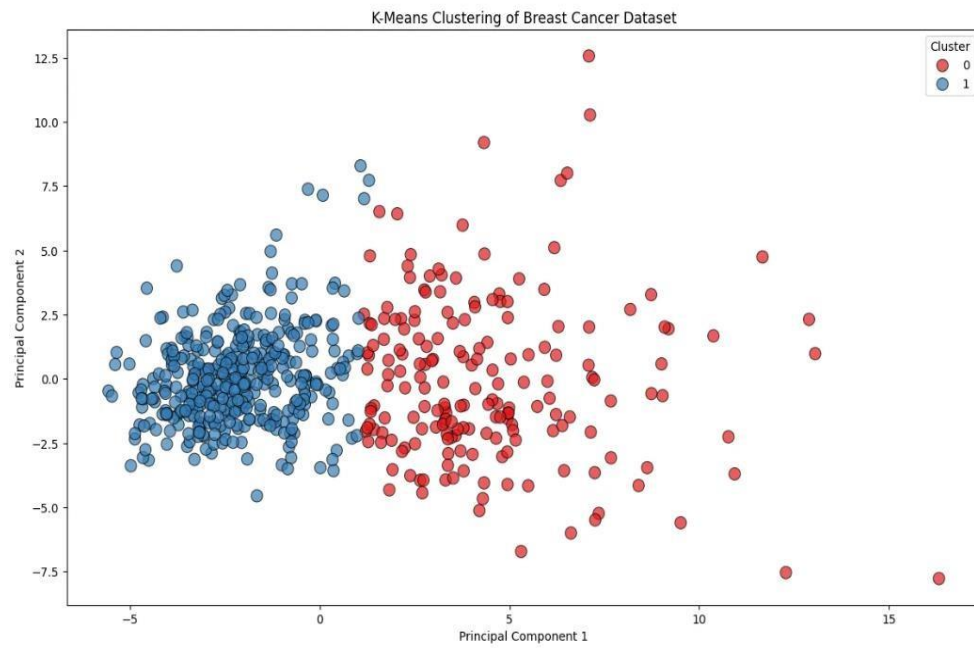
# Set optimal number of clusters
optimal_k = 2
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_pca)

# Visualize the Clusters
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap="viridis", alpha=0.6)
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1], s=200, c='red',
            marker='X', label='Centroids')
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clustering on Breast Cancer Dataset")
plt.legend()
plt.show()

```

OUTPUT:





CONTENT BEYOND SYLLABUS

1. Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .CSV file.

```
import csv
with open('tennis.csv', 'r') as f:
    reader = csv.reader(f)
    your_list = list(reader)
    h = [['0', '0', '0', '0', '0', '0']]
    for i in your_list:
        print(i)
        if i[-1] == "True":
            j = 0
            for x in i:
                if x != "True":
                    if x != h[0][j] and h[0][j] == '0':
                        h[0][j] = x
                    elif x != h[0][j] and h[0][j] != '0':
                        h[0][j] = '?'
            else:
                pass
            j = j + 1
        print("Most specific hypothesis is")
        print(h)
    Output
    'Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', True
    'Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', True
    'Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', False
    'Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', True
    Maximally Specific set
    [['Sunny', 'Warm', '?', 'Strong', '?', '?']]
```

2. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

```
import numpy as np
X = np.array([2, 9], [1, 5], [3, 6]), dtype=float)
y = np.array([92], [86], [89]), dtype=float)
X = X/np.amax(X,axis=0) # maximum of X array longitudinally
y = y/100
#Sigmoid Function
def sigmoid (x):
    return 1/(1 + np.exp(-x))
```

```

#Derivative of Sigmoid Function
def derivatives_sigmoid(x):
    return x * (1 - x)
#Variable initialization
epoch=7000 #Setting training iterations
lr=0.1 #Setting learning rate
inputlayer_neurons = 2 #number of features in data set
hiddenlayer_neurons = 3 #number of hidden layers neurons
output_neurons = 1 #number of neurons at output layer
#weight and bias initialization
wh=np.random.uniform(size=(inputlayer_neurons,hiddenlayer_neurons))
bh=np.random.uniform(size=(1,hiddenlayer_neurons))
wout=np.random.uniform(size=(hiddenlayer_neurons,output_neurons))
bout=np.random.uniform(size=(1,output_neurons))
#draws a random range of numbers uniformly of dim x*y
for i in range(epoch):
    #Forward Propagation
    hinp1=np.dot(X,wh)
    hinp=hinp1 + bh
    hlayer_act = sigmoid(hinp)
    outinp1=np.dot(hlayer_act,wout)
    outinp= outinp1+ bout
    output = sigmoid(outinp)
    #Backpropagation
    EO = y-output
    outgrad = derivatives_sigmoid(output)
    d_output = EO* outgrad
    EH = d_output.dot(wout.T)
    hiddengrad = derivatives_sigmoid(hlayer_act)#how much hidden layer wts
    contributed to error
    d_hiddenlayer = EH * hiddengrad
    wout += hlayer_act.T.dot(d_output) *lr# dotproduct of nextlayererror and
    currentlayerop
    # bout += np.sum(d_output, axis=0,keepdims=True) *lr
    wh += X.T.dot(d_hiddenlayer) *lr
    #bh += np.sum(d_hiddenlayer, axis=0,keepdims=True) *lr
    print("Input: \n" + str(X))
    print("Actual Output: \n" + str(y))
    print("Predicted Output: \n" ,output)

output
Input:
[[ 0.66666667  1. ]
 [ 0.33333333  0.55555556]]

```

```
[ 1. 0.66666667]]
```

Actual Output:

```
[[ 0.92]
```

```
[ 0.86]
```

```
[ 0.89]]
```

Predicted Output:

```
[[ 0.89559591]
```

```
[ 0.88142069]
```

```
[ 0.8928407 ]
```

3. Assuming a set of documents that need to be classified, use the naïve BayesianClassifier model to perform this task. Built-in Java classes/API can be used to write the program. Calculate the accuracy, precision, and recall for your data set.

```
import pandas as pd
msg=pd.read_csv('naivetext1.csv',names=['message','label'])
print('The dimensions of the dataset',msg.shape)
msg['labelnum']=msg.label.map({'pos':1,'neg':0})
X=msg.message
y=msg.labelnum
print(X)
print(y)
#splitting the dataset into train and test data
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest=train_test_split(X,y)
print(xtest.shape)
print(xtrain.shape)
print(ytest.shape)
print(ytrain.shape)
#output of count vectoriser is a sparse matrix
from sklearn.feature_extraction.text import CountVectorizer
count_vect = CountVectorizer()
xtrain_dtm = count_vect.fit_transform(xtrain)
xtest_dtm=count_vect.transform(xtest)
print(count_vect.get_feature_names())
df=pd.DataFrame(xtrain_dtm.toarray(),columns=count_vect.get_feature_names())
print(df)#tabular representation
print(xtrain_dtm) #sparse matrix representation
# Training Naive Bayes (NB) classifier on training data.
from sklearn.naive_bayes import MultinomialNB
clf = MultinomialNB().fit(xtrain_dtm,ytrain)
predicted = clf.predict(xtest_dtm)
#printing accuracy metrics
from sklearn import metrics
```



```

print('Accuracy metrics')
print('Accuracy of the classifier is',metrics.accuracy_score(ytest,predicted))
print('Confusion matrix')
print(metrics.confusion_matrix(ytest,predicted))
print('Recall and Precision ')
print(metrics.recall_score(ytest,predicted))
print(metrics.precision_score(ytest,predicted))
'''docs_new = ['I like this place', 'My boss is not my saviour']
X_new_counts = count_vect.transform(docs_new)
predictednew = clf.predict(X_new_counts)
for doc, category in zip(docs_new, predictednew):
print('%s->%s' % (doc, msg.labelnum[category]))'''
I love this sandwich,pos
This is an amazing place,pos
I feel very good about these beers,pos
This is my best work,pos
What an awesome view,pos
I do not like this restaurant,neg
I am tired of this stuff,neg
I can't deal with this,neg
He is my sworn enemy,neg
My boss is horrible,neg
This is an awesome place,pos
I do not like the taste of this juice,neg
I love to dance,pos
I am sick and tired of this place,neg
What a great holiday,pos
That is a bad locality to stay,neg
We will have good fun tomorrow,pos
I went to my enemy's house today,neg

```

OUTPUT

```

['about', 'am', 'amazing', 'an', 'and', 'awesome', 'beers', 'best', 'boss', 'can', 'deal',
'do', 'enemy', 'feel', 'fun', 'good', 'have', 'horrible', 'house', 'is', 'like', 'love', 'my',
'not', 'of', 'place', 'restaurant', 'sandwich', 'sick', 'stuff', 'these', 'this', 'tired', 'to',
'today', 'tomorrow', 'very', 'view', 'we', 'went', 'what', 'will', 'with', 'work']
about am amazing an and awesome beers best boss can ... today \
0 1 0 0 0 0 0 1 0 0 0 ... 0
1 0 0 0 0 0 0 0 1 0 0 ... 0
2 0 0 1 1 0 0 0 0 0 0 ... 0
3 0 0 0 0 0 0 0 0 0 0 ... 1
4 0 0 0 0 0 0 0 0 0 0 ... 0
5 0 1 0 0 1 0 0 0 0 0 ... 0
6 0 0 0 0 0 0 0 0 0 1 ... 0
7 0 0 0 0 0 0 0 0 0 0 ... 0

```

80100000000...0
90001010000...0
100000000000...0
1100000000010...0
120001010000...0
tomorrow very view we went what will with work
0010000000
1000000001
2000000000
3000010000
4000000000
5000000000
60000000010
7100100100
8000000000

VIVA QUESTIONS

1. What is Machine Learning?
2. How does Machine Learning differ from traditional programming?
3. What are different types of Machine Learning?
4. What are some popular algorithms used in Machine Learning?
5. How do you handle overfitting in Machine Learning?
6. What is Bias-Variance Tradeoff in Machine Learning?
7. How would you evaluate the performance of the Machine Learning model?
8. What is feature selection, and why is it important?
9. How do you handle missing values in a dataset?
10. How can Machine Learning be applied in real-world scenarios?